Python

Unidade I – Fundamentos
Seção 3 – Primeiros passos

Primeiro Passos

Primeiros Passos

- Neste material, começaremos a nos familiarizar com a escrita e execução de códigos Python.

Hello ?

- “**Hello World**” é um marco inicial na aprendizagem de qualquer linguagem de programação.
- Ele consiste em exibir uma mensagem simples na tela, demonstrando a estrutura básica de escrita e execução de um programa.

```
print("Hello, World!")
```

Simplicidade e legibilidade

- Esse exemplo evidencia a **simplicidade** e **legibilidade** da linguagem Python.
- O comando print() faz parte da **biblioteca padrão**, responsável por enviar o conteúdo especificado aos fluxos de saída, normalmente a tela do console ou terminal.

Dica:

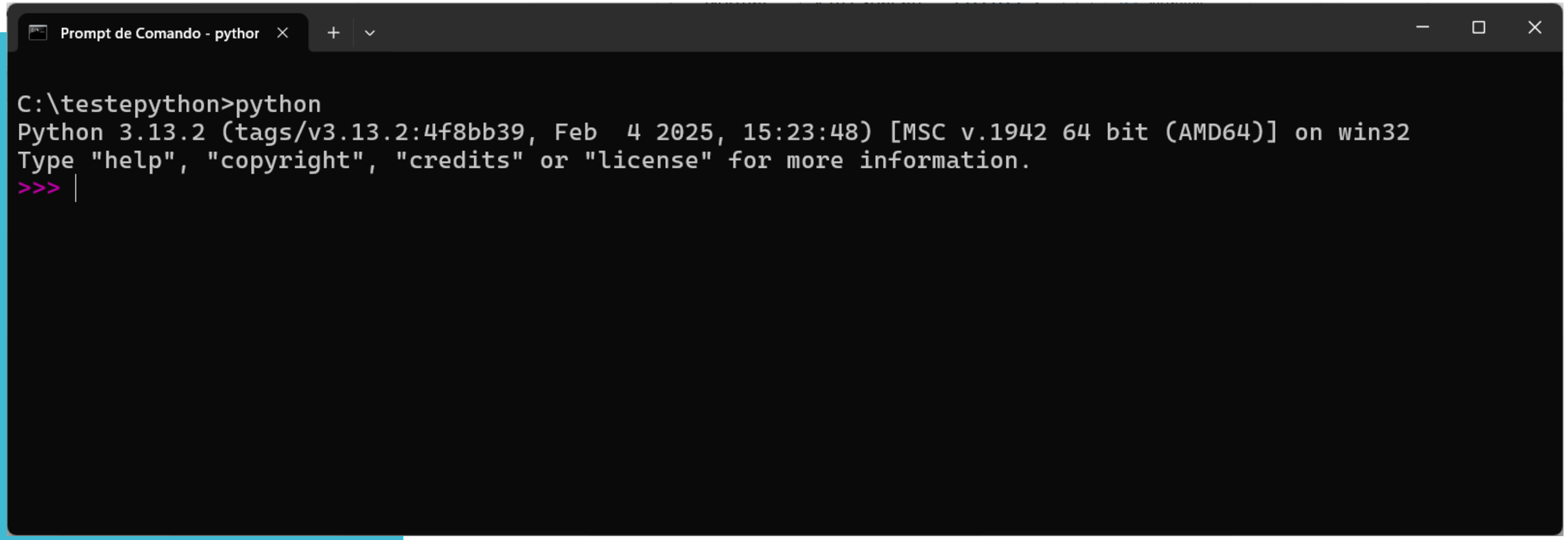
- **Dica**: Em Python 3, print é uma função.
- Caso você já tenha visto exemplos em Python 2, notará que a sintaxe difere levemente, pois print costumava ser uma instrução e não exigia parênteses.

Interpretador Python Interativo

- O **interpretador interativo** do Python é uma ferramenta essencial para experimentação e aprendizado.
- Ele permite testar **pequenos trechos de código** sem precisar criar arquivos .py, oferecendo feedback imediato sobre o comportamento do código.

Acessando o Interpretador

1. **Abra o Terminal ou Prompt de Comando**:
   Dependendo do seu sistema operacional (Windows, Linux ou macOS).
2. **Digite**:
   python
- Pressione enter



Esse **>>>** é o **prompt do interpretador**, indicando que você pode inserir comandos em Python interativamente.

Experimentando no Interpretador

Qualquer expressão ou comando Python digitado será imediatamente executado. Exemplos:

```
>>> 2 + 2
4
>>> "Olá " + "Python"
'Olá Python'
>>> nome = "Maria"
>>> print("Bem-vinda,", nome)
Bem-vinda, Maria
```

```
Prompt de Comando - pythor

C:\testepython>python
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2+2
4
>>> "ola " + " Maromo"
'ola  Maromo'
>>> nome = "Maromo"
>>> saudacao = "ola " + nome
>>> print(saudacao)
ola Maromo
>>>
```

Esses exemplos permitem **testar hipóteses**, **executar cálculos rápidos** ou **relembrar sintaxes** sem a necessidade de criar scripts separados. Para sair do interpretador interativo, basta pressionar Ctrl + D (no Linux e macOS) ou Ctrl + Z e depois Enter (no Windows), ou ainda digitar exit().

Scripts e Execução Via Terminal

- Quando o programa excede alguns trechos de teste ou necessita de maior permanência, criamos scripts Python – arquivos de texto puro geralmente com extensão .py.
- Eles podem ser executados direto no terminal, sendo úteis para automações, projetos de médio porte e aplicações mais complexas.

Criando um Arquivo de Script

1. **Abra seu editor ou IDE (VS Code)**.
2. **Crie um arquivo chamado** exemplo.py (ou outro nome significativo).
3. **Insira o código**:

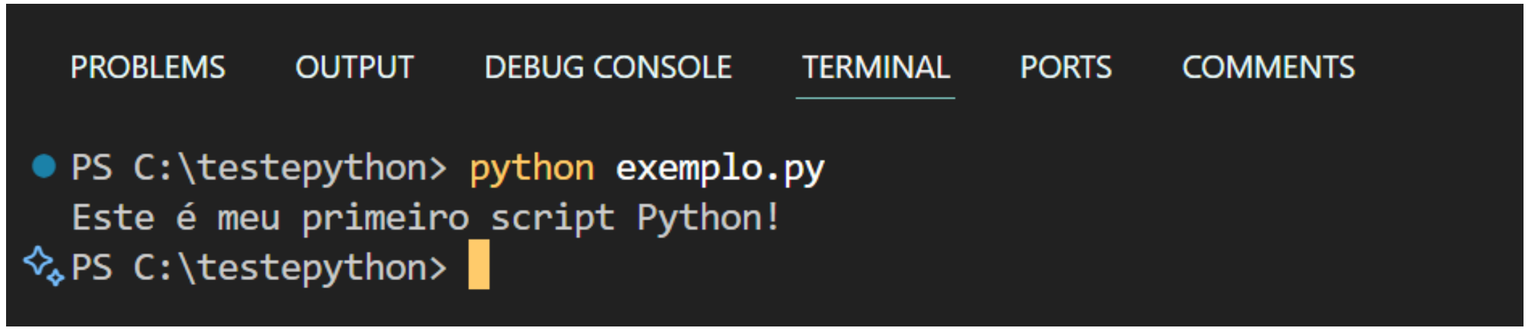
```
print("Este é meu primeiro script Python!")
```

Exemplo

File
Edit
Selection
View
testepython
EXPLORER
TESTEPYTHON
exemplo.py
exemplo.py
Extension: Pylance
exemplo.py
1
print("Este é meu primeiro script Python!")

Executando o Script

No terminal, navegue até o diretório onde o arquivo foi salvo ou CTRL + ‘ (apóstrofo) no VS Code.
Por exemplo:
python exemplo.py
A saída esperada será:
Este é meu primeiro script Python!



Estrutura de um Script

- Embora um script simples possa conter apenas algumas linhas, projetos maiores frequentemente contam com:
  - **Importações de bibliotecas** (via import algo).
  - **Declaração de variáveis** e funções.
  - **Bloco principal** (if __name__ == "__main__":), que indica o ponto de partida do script.

Exemplo com função main

```
def main():
    # Aqui você escreve a lógica principal do programa
    print("Executando a função principal...")

if __name__ == "__main__":
    main()
```

Comentários e Documentação Básica

- Em projetos reais, **comentários** e **documentação** interna são cruciais para manter o código compreensível e facilitar futuras modificações ou revisões. Python oferece duas principais formas de incluir comentários:
  1. Comentários de Linha Única
  2. Docstrings (Comentários em Múltiplas Linhas)

Comentários de uma linha

```
# Esta função imprime uma saudação
def saudar():
        print("Olá!")
```

Esses comentários **não** são executados pelo interpretador; servem apenas para esclarecimentos ao leitor do código.

Docstrings

- Docstrings são usadas para fornecer explicações mais detalhadas sobre **funções**, **módulos** ou **classes**. Elas são definidas entre aspas triplas (""" ... """) e podem abranger múltiplas linhas.

```
def soma(a, b):
    """
    Retorna a soma de dois números.

    Parâmetros:
    a (int ou float) - Primeiro número.
    b (int ou float) - Segundo número.

    Retorno:
    int ou float - Resultado da soma de a e b.
    """
    return a + b
```

```
exemplo3.py > main
1   def soma(a, b
10      int ou fl
11      """
12      return a
13
    Codeium: Refactor
14  def main():
15      x = soma()
16      print(x)
17
18  if __name__ == "__main__":
19      main()
```

```
(a: Any, b: Any) -> Any

a (int ou float) – Primeiro número.

Retorna a soma de dois números.

Parâmetros: a (int ou float) – Primeiro número. b (int ou float) – Segundo número.   Retorno: int ou float – Resultado da soma de a e b.
```

Na linha 15, ao digitar a palavra soma e abrir os parênteses o VS Code já apresenta o que foi colocado no Docstring.

Acessando Docstrings

- **help()**
  - No interpretador interativo, o comando help() exibe a docstring associada a uma função.
- **Exemplo:**
  - >>> help(soma)
- **Importando funções**
  - Caso a função esteja em um arquivo .py, importe-a antes de chamar help().
- **Exemplo:**
  - >>> from exemplo3 import soma
  - >>> help(soma)
- **Onde exemplo3 é o nome do script (sem a extensão .py).**

Resultado

```
>>> from exemplo3 import soma
>>> help(soma)
Help on function soma in module exemplo3:

soma(a, b)
        Retorna a soma de dois números.

        Parâmetros:
        a (int ou float) \u2013 Primeiro número.
        b (int ou float) \u2013 Segundo número.

        Retorno:
        int ou float \u2013 Resultado da soma de a e b.

>>>
```

Vantagem do Docstrings

- Docstrings tornam o código **autodescritivo**.
- Ferramentas de documentação automática (como **Sphinx**) podem usá-las para gerar manuais ou sites de documentação completos do seu projeto.

Conclusão

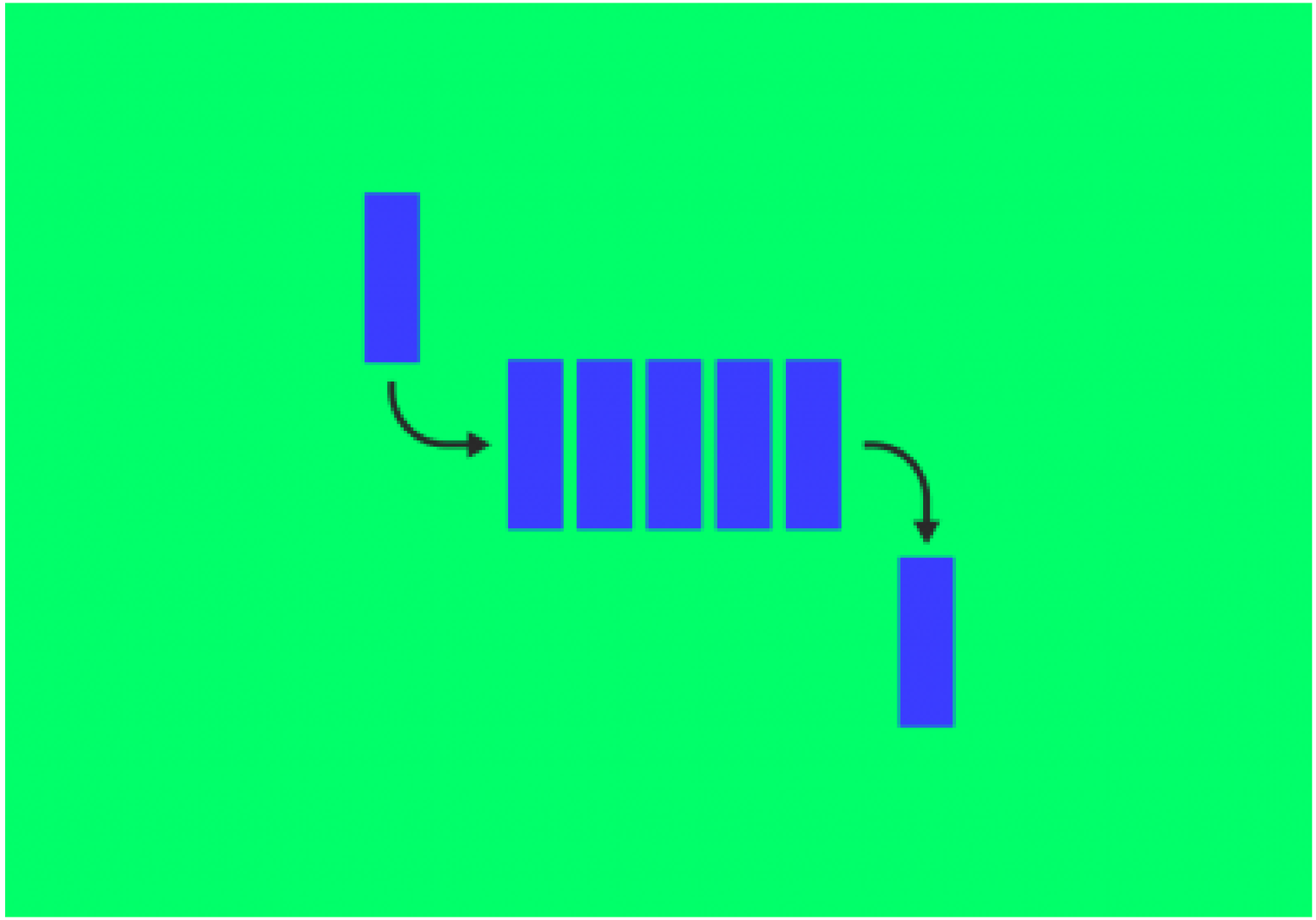
Nesta seção, você aprendeu:

1. **Hello World**: O exemplo clássico que demonstra a sintaxe simples do Python.
2. **Interpretador Interativo**: Uma forma imediata de testar códigos curtos e entender o comportamento de funções e expressões.
3. **Scripts Python**: Como criar um arquivo .py e executá-lo via terminal para programas de maior escopo.

Python

Unidade II – A Linguagem Python e Seus Recursos
Seção 4 - Estruturas de Dados e Tipos

Estruturas de Dados e Tipos



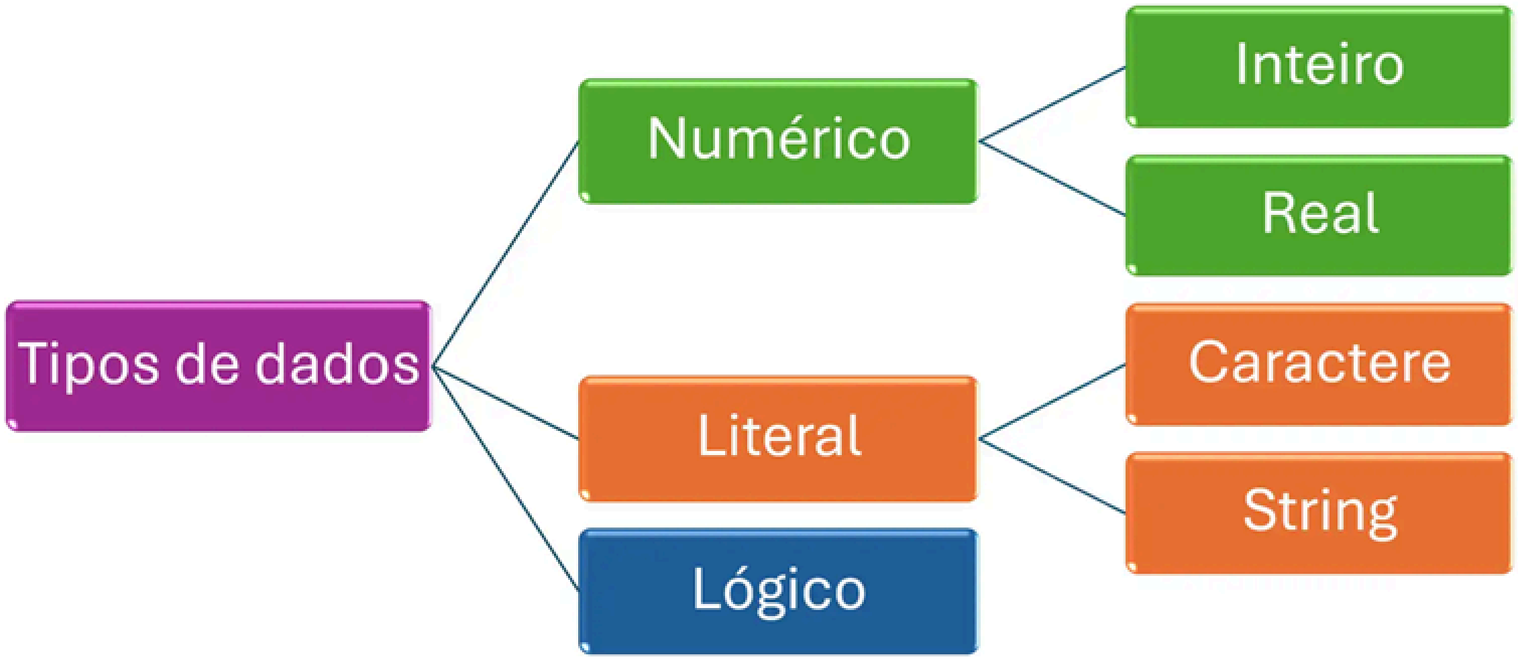
Estrutura de Dados e Grafos

- Estruturas de dados são fundamentais para a eficiência e organização de algoritmos e grafos em Python.
- A escolha da estrutura de dados correta pode impactar significativamente o desempenho de um algoritmo, especialmente em algoritmos que manipulam grandes conjuntos de dados ou quando a eficiência é crítica.
- **Em grafos, estruturas de dados como listas de adjacência e matrizes de adjacência são usadas para representar as conexões entre os nós (vértices) e arestas**, e a escolha entre elas afeta a forma como as operações de busca e manipulação são realizadas.

Estrutura de Dados e Tipos

- Nesta seção, aprofundaremos nosso conhecimento sobre os tipos de dados básicos e as estruturas de dados oferecidos pelo Python, elementos essenciais para a manipulação, armazenamento e organização de informações em qualquer projeto de software.
- Abordaremos os tipos numéricos, lógicos e textuais, além de apresentar estruturas compostas como listas, tuplas, dicionários e conjuntos (sets).
- Também discutiremos características avançadas de cada tipo, boas práticas e exemplos de uso.

Tipos Básicos



Tipos Básicos

Python fornece uma série de tipos de dados primitivos que atendem a diversas necessidades – desde operações aritméticas até processamento de texto e manipulação de valores lógicos.

Inteiros

- Representam números inteiros (positivos, negativos ou zero).
- Em Python 3, os inteiros têm precisão ilimitada, significando que podem crescer além dos limites tradicionais de 32 ou 64 bits (dependendo apenas da memória disponível).
- Exemplos de criação:

```
numero = 10
saldo = -500
print(numero)  # 10
print(saldo)   # -500
```

Operações Comuns

Soma, subtração, multiplicação, divisão inteira, resto de divisão.

```
a = 15
b = 4
print(a + b)    # 19
print(a // b)   # 3 (divisão inteira)
print(a % b)    # 3 (resto)
print(a ** b)   # 15^4 = 50625 (exponenciação)
```

Ponto Flutuante

- Representam **números com casas decimais**.
- Geralmente, o Python utiliza o padrão de ponto flutuante em **dupla precisão** (64 bits), semelhante a double em C/C++.
- **Exemplos de criação:**

```
altura = 1.75
preco = 19.99
print(altura)  # 1.75
print(preco)   # 19.99
```

Cuidado com floats

- Arredondamentos e imprecisões devido à forma como números decimais são representados em binário.
- Exemplo:

```
print(0.1 + 0.2)  # 0.30000000000000004 (imprecisão)
```

Para necessidades de **alta precisão** (ex.: cálculos financeiros), considere módulos como decimal ou fractions.

Função	Exemplo (com x = 81, y = 3.14159)	Resultado do Exemplo	Descrição
abs()	abs(-5)	5	Retorna o valor absoluto de um número (remove o sinal negativo, se houver).
round()	round(y, 2)	3.14	Arredonda um número para um número específico de casas decimais. O primeiro argumento é o número, o segundo é a quantidade de casas.
int()	int(y)	3	Converte um valor para um número inteiro, truncando a parte decimal.
float()	float(10)	10.0	Converte um valor para um número de ponto flutuante (decimal).
max()	max(10, 5, 20)	20	Retorna o maior valor de uma sequência de números ou argumentos.
min()	min(10, 5, 20)	5	Retorna o menor valor de uma sequência de números ou argumentos.
sum()	sum([1, 2, 3])	6	Retorna a soma de todos os itens em um iterável (como uma lista ou tupla) de números.
pow()	pow(5, 3)	125	Calcula a potência de um número (base elevado ao expoente). É equivalente ao operador **.
math.sqrt()	math.sqrt(x)	9.0	Calcula a raiz quadrada de um número. Importante: Requer import math no início do seu código.
math.ceil()	math.ceil(y)	4	Retorna o menor inteiro maior ou igual a um número (arredonda para cima). Requer import math.
math.floor()	math.floor(y)	3	Retorna o maior inteiro menor ou igual a um número (arredonda para baixo). Requer import math.
math.trunc()	math.trunc(y)	3	Retorna a parte inteira do número, removendo a parte fracionária (sem arredondar). Requer import math.

Booleanos

- Representam valores lógicos: **True** ou **False** (com inicial maiúscula no Python).
- São úteis em avaliações de condições e controle de fluxo.
- **Exemplos de criação:**

```
ativo = True
permitido = False
print(ativo)      # True
print(permitido) # False
```

- **Operadores lógicos**: and, or, not.

```
print(True and False)  # False
print(True or False)   # True
print(not True)        # False
```

Strings

- Representam **textos** (sequências de caracteres).
- Podem ser definidas com aspas simples ('...') ou duplas ("...").
- **Exemplos:**

```
nome = "Python"
mensagem = 'Olá, Mundo!'
print(nome)       # Python
print(mensagem)   # Olá, Mundo!
```

Nota sobre Strings

- **Imutabilidade**: Strings em Python são **imutáveis**, ou seja, não podem ser alteradas após criação. Qualquer “modificação” gera uma nova string.
- **Acesso a caracteres** (indexação) e slices:

```
linguagem = "Python"
print(linguagem[0])    # 'P'
print(linguagem[2:5])  # 'tho'
```

Métodos para Manipular Strings

Métodos	Descrição
upper()	Converte todos os caracteres da string para letras maiúsculas.
lower()	Converte todos os caracteres da string para letras minúsculas.
capitalize()	Converte o primeiro caractere da string para maiúscula e o restante para minúscula.
strip()	Remove espaços (ou outros caracteres especificados) do início e do final da string.
replace()	Substitui todas as ocorrências de uma substring por outra substring na string.
split()	Divide a string em uma lista de substrings, usando um separador (delimitador).

Métodos úteis. Exemplos

```
frase = "  Olá, Python!  "
print(frase.strip())        # "Olá, Python!"
print(frase.replace("Olá", "Hello"))  #  “  Hello,Python!”
print(frase.upper())      # “HELLO, PYTHON !”
print(frase.lower()) # “hello, python !”
```

Nota

Exercícios de Python para Revisão: Engenharia de Computação no Anexo do Material: Anexo02_Material02.pdf

Coleções em Python

Collections

O que são Coleções?

- Em Python, **coleções** (também conhecidas como estruturas de dados) são como "contêineres" que nos permitem armazenar e organizar múltiplos valores juntos.
- Imagine que você tem vários itens relacionados – talvez uma lista de compras, um conjunto de números, ou informações sobre um produto. Em vez de criar uma variável para cada item, as coleções nos dão maneiras eficientes de agrupar esses dados.

Por que usar Coleções?

- **Organização:**
  - Agrupar dados que pertencem um ao outro.
- **Eficiência:**
  - Manipular vários itens de uma vez.
- **Flexibilidade:**
  - Cada tipo de coleção tem características únicas, permitindo escolher a melhor para cada situação.

Principais Tipos de Coleções em Python

- **Listas (list):**
- **Tuplas (tuple):**
- **Conjuntos (set):**
- **Dicionários (dict):**

Listas

- **Coleção ordenada e mutável** de elementos.
- Definidas entre colchetes [ ].
- **Aceitam** valores de tipos diferentes no mesmo container.
- Exemplos:

```
frutas = ["maçã", "banana", "cereja"]
numeros = [1, 2, 3, 4, 5]
print(frutas)   # ['maçã', 'banana', 'cereja']
print(numeros)  # [1, 2, 3, 4, 5]
```

```
print(frutas[0])     # 'maçã'
print(frutas[-1])    # 'cereja' (último elemento)
print(frutas[1:3])   # ['banana', 'cereja']
```

Listas: Principais métodos

- **`append(item)`:**
  - Adiciona item ao final.
- **`remove(item)`:**
  - Remove a primeira ocorrência de item.
- **`pop([índice])`:**
  - Remove e retorna o elemento de determinado índice (padrão: último).
- **`sort()`:**
  - Ordena a lista (in-place).
- **`reverse()`:**
  - Inverte a ordem da lista (in-place).

Listas
Exemplos

```
frutas.append("uva")
print(frutas)  # ['maçã', 'banana', 'cereja', 'uva']

frutas.remove("banana")
print(frutas)  # ['maçã', 'cereja', 'uva']

ultimo = frutas.pop()
print(ultimo)  # 'uva'
print(frutas)  # ['maçã', 'cereja']
```

Tuplas

- **Coleção ordenada e imutável** de elementos.
- Definidas entre parênteses ( ).
- Assemelham-se a listas, exceto pela **imutabilidade**: uma vez criada, a tupla não pode ser modificada (não aceita **append, remove**, etc.).
- **Exemplos:**

```
coordenadas = (10.0, 20.0)
dias_semana = ("Segunda", "Terça", "Quarta")
print(coordenadas)  # (10.0, 20.0)
print(dias_semana)  # ('Segunda', 'Terça', 'Quarta')
```

Tuplas: Uso típico

- **Uso típico**: armazenar valores que não devem ser alterados, ou representar estruturas fixas, como (latitude, longitude), etc.
- **Desempacotamento**:

```
x, y = coordenadas
print(x)  # 10.0
print(y)  # 20.0
```

Conjuntos (set):

- Coleções **não ordenadas** de itens únicos (não aceitam duplicatas).
  - Exemplo: {1, 2, 3, 3} resultaria em {1, 2, 3}

Conjuntos
Uso típico

- **Uso típico:** A principal força de um set reside em sua capacidade de lidar com exclusividade e operações de conjunto. Aqui estão os usos mais comuns:
  - 1. Remover Elementos Duplicados de uma Lista (uso mais frequente)
  - 2. Verificar a Existência de um Elemento Rapidamente
  - 3. Realizar operações de Conjunto (Matemática)
  - 4. Filtrar Itens Baseado em Exclusividade

Conjunto: Exemplos

```
numeros_duplicados = [1, 2, 2, 3, 4, 4, 5, 1]
numeros_unicos = list(set(numeros_duplicados))
print(numeros_unicos)
```

```
emails_validos = {"usuario1@email.com",
"usuario2@email.com", "admin@empresa.com"}
if "usuario1@email.com" in emails_validos:
    print("Email encontrado!")
else:
    print("Email não encontrado.")
```

Conjunto: Exemplos (2)

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.union(b))  # ou print(a | b)
# Saída: {1, 2, 3, 4, 5}
```

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.intersection(b)) # ou print(a & b)
# Saída: {3}
```

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.difference(b)) # ou print(a - b)
# Saída: {1, 2}
print(b.difference(a)) # ou print(b - a)
# Saída: {4, 5}
```

Conjuntos: adição e remoção

```
numeros_unicos.add(4)
numeros_unicos.remove(2)
print(numeros_unicos)  # {1, 3, 4}
```

Dicionários

- **Estruturas de mapeamento** que armazenam pares chave-valor.
- Definidos entre chaves { }.
- Chaves (keys) devem ser **hasháveis** (tipicamente strings, números ou tuplas), enquanto os valores podem ser qualquer objeto Python.
- Exemplos:

```
usuario = {"nome": "Alice", "idade": 30}
estoque = {"maçã": 50, "banana": 100}
print(usuario)  # {'nome': 'Alice', 'idade': 30}
print(estoque)  # {'maçã': 50, 'banana': 100}
```

Conclusão

- **keys():**
  - é utilizado para iterar apenas sobre as chaves do dicionário.
- **values():**
  - itera apenas sobre os valores.
- **items():**
  - retorna os pares (chave, valor), o que é útil para acessar e manipular simultaneamente a chave e o valor.

Acesso via chave

```
print(usuario["nome"])  # 'Alice'
# Ou usando get():
print(usuario.get("idade"))  # 30
```

Modificação:

```
usuario["cidade"] = "São Paulo"  # Adiciona nova chave-valor
usuario["idade"] = 31            # Altera valor existente
print(usuario)
```

Métodos comuns

- keys(), values(), items() para iterar sobre chaves, valores ou ambos.
- pop(chave) remove e retorna o valor associado à chave.

```
# Criação de um dicionário de exemplo
pessoas = {
    "Marcos": 35,
    "Ana": 28,
    "João": 40
}

# Iterando sobre as chaves com keys()
print("Chaves do dicionário:")
for chave in pessoas.keys():
    print(chave)
```

Exemplo de values

```
# Utilizando o mesmo dicionário de exemplo
pessoas = {
    "Marcos": 35,
    "Ana": 28,
    "João": 40
}

# Iterando sobre os valores com values()
print("Valores do dicionário:")
for valor in pessoas.values():
    print(valor)
```

Exemplo de items

```
# Utilizando o mesmo dicionário
pessoas = {
    "Marcos": 35,
    "Ana": 28,
    "João": 40
}

# Iterando sobre chaves e valores com items()
print("Pares (chave, valor) do dicionário:")
for chave, valor in pessoas.items():
    print(f"Chave: {chave}, Valor: {valor}")
```

Tabela Comparativa: Coleções em Python

- Para facilitar a visualização das diferenças e escolher a ferramenta certa para cada situação, no próximo slide temos uma tabela comparativa detalhada das quatro principais estruturas de coleções em Python: **Listas, Tuplas, Conjuntos (Sets) e Dicionários.**

Característica Principal	Lista (list)	Tupla (tuple)	Conjunto (set)	Dicionário (dict)
Definição	Coleção ordenada e mutável de itens.	Coleção ordenada e imutável de itens.	Coleção não ordenada de itens únicos e mutáveis.	Coleção não ordenada de pares chave-valor e mutáveis.
Sintaxe	Colchetes []	Parênteses () (ou sem nada para tuplas simples)	Chaves {} (com itens) ou set() para vazio	Chaves {} (com pares chave-valor)
Mutável?	Sim (pode adicionar, remover, modificar)	Não (imutável após a criação)	Sim (pode adicionar, remover itens)	Sim (pode adicionar, remover, modificar pares)
Ordenado?	Sim (mantém a ordem de inserção)	Sim (mantém a ordem de inserção)	Não (a ordem dos itens não é garantida)	Sim (a partir do Python 3.7+ mantém a ordem de inserção das chaves)
Indexável?	Sim (acesso por índice numérico [i])	Sim (acesso por índice numérico [i])	Não (itens não têm uma ordem fixa)	Sim (acesso por chave [chave])
Permite Duplicatas?	Sim	Sim	Não (itens duplicados são automaticamente removidos)	Não (chaves devem ser únicas, valores podem ser duplicados)
Uso Típico	Lista de compras, histórico de eventos, filas, pilhas.	Coordenadas geográficas, registro de dados fixos, retorno de múltiplas informações de uma função.	Remover duplicatas, operações de conjuntos (união, interseção), verificar presença rápida.	Informações de perfil (nome, idade), configurações, mapeamento de IDs para objetos.
Exemplo	[1, "a", True]	(1, "b", False)	{1, 2, 3}	{"nome": "Maria", "idade": 25}

Conversão entre Tipos

Conversões Entre Tipos

- Em Python, é comum **converter** valores entre diferentes tipos, principalmente ao manipular dados oriundos de arquivos ou de inputs do usuário.
  - **int()**, **float()**: Convertem strings ou valores numéricos para inteiros ou floats.
  - **str()**: Converte qualquer objeto Python em string.
  - **list(), tuple(), set()**: Transformam coleções em outros formatos.

Conversão: Exemplos

```
texto = "123"
numero_inteiro = int(texto)  # 123 (int)
numero_float = float(texto)  # 123.0 (float)

lista_nums = list({1, 2, 3})
# [1, 2, 3] (ordem pode variar pois {set} não é ordenado)
tupla_nums = tuple(lista_nums) # (1, 2, 3)
```

Boas Práticas

Escolher a Estrutura Certa:

- **Listas**: Se precisa de ordem e mutabilidade.
- **Tuplas**: Se precisa de ordem, mas não de mutabilidade.
- **Dicionários**: Se deseja relacionar chaves e valores.
- **Conjuntos**: Se busca eliminar duplicatas e precisa de operações matemáticas de conjunto.

Boas Práticas

Nomear Variáveis de Forma Significativa:
Evite nomes genéricos (ex.: a, b, c). Prefira nomes que expressam a intenção do valor (ex.: lista_frutas, estoque_produtos).
Aproveitar Métodos Internos:
Funções como append, pop, update, union, etc. tornam o código mais limpo e eficiente.
Imutabilidade Planejada:
Se a coleção não deve mudar, use **tuplas** ou **namedtuples**. Isso evita bugs e facilita a manutenção.

Recapitulando

- Nesta seção, vimos como o Python oferece tipos básicos – inteiros, floats, booleanos e strings – e estruturas de dados poderosas – listas, tuplas, dicionários e conjuntos – para representar praticamente qualquer tipo de informação necessária em um programa.
- Compreender as características, vantagens e limitações de cada tipo/estrutura é fundamental para escrever códigos mais claros, eficientes e confiáveis.
- Compreendemos a importância das estruturas de dados para o estudo de grafos.

Exercícios

- Para prática da aula, acesse o Material:
  - **Anexo03_Material02.pdf**

FIM